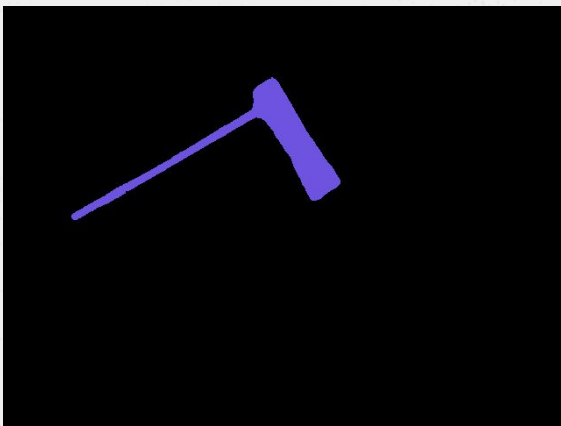


Picking Point

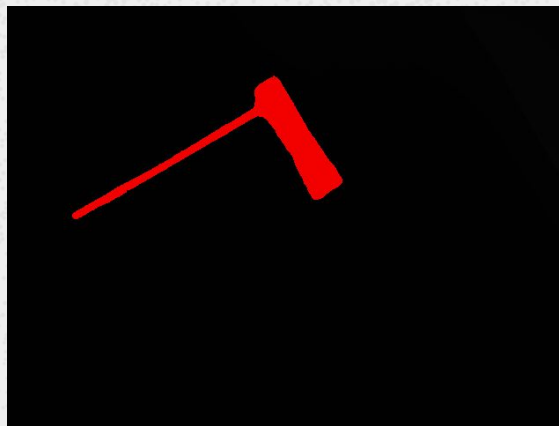
Fabio Scappi and Federico Buoncuore

Inputs and outputs

Our program takes as input a mask for an object and a masked depth map. The program outputs various images representing the steps involved in finding the picking point, the picking point itself, and the required opening and rotation angle for the clamp. Below, you can see an example of the input.



The mask

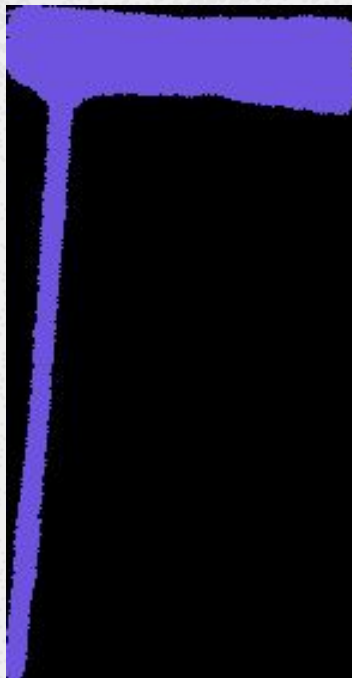


The depth (in EXR format)

Cropping the images

As you can see, the object only covers a small part of the image, so before starting we crop the images to extract the object.

We also rotate the object to simplify the calculations.



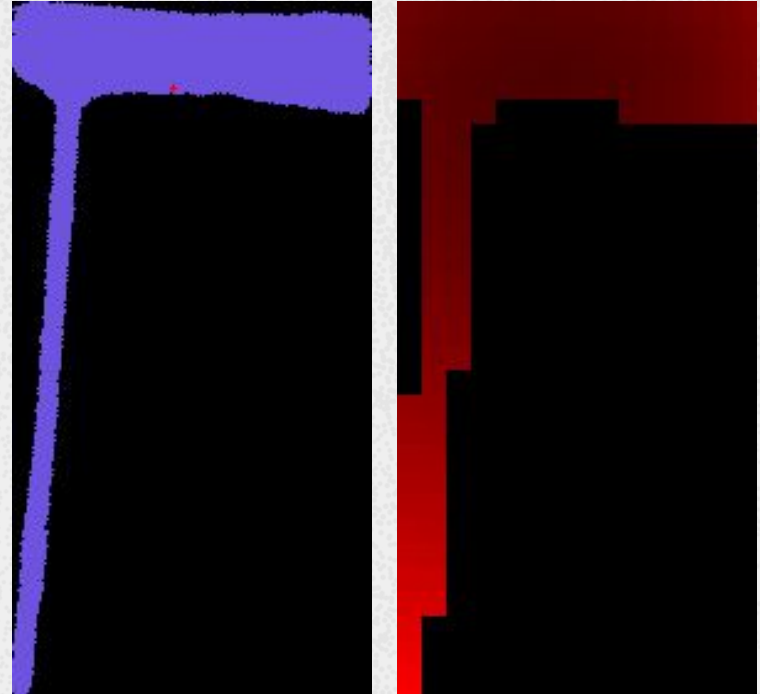
Dividing in cells

We begin by dividing the image into larger cells. Each cell is then assigned a score based on the density of cells surrounding it. The optimal picking point is located in the surroundings of the area with the highest object density, which in our case is where there are the darker cells.



The first point

To find the first point we sort the cells in ascending order, and we take the first one. This is not the final picking point, but we know that the final picking point will be near this first point.



Finding the optimal point

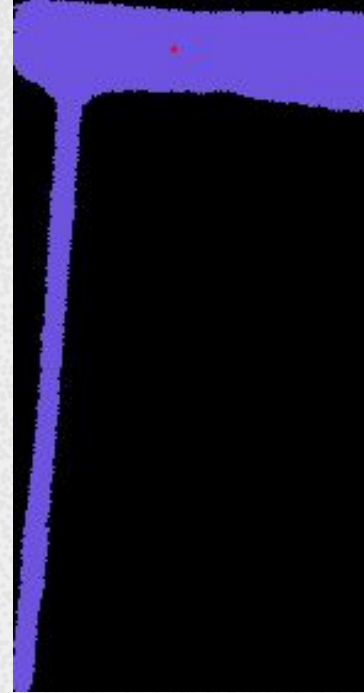
From the first point we cast 4 rays, that will stop in 2 cases:

- The ray hits a black pixel (the background)
- The difference between the depth of the current pixel and the depth of the last pixel is greater than a threshold



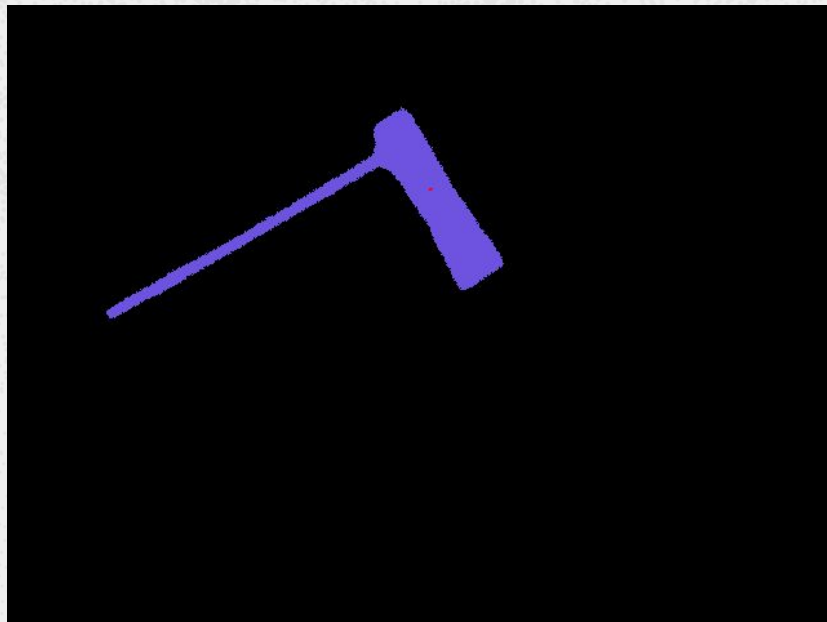
Finding the optimal point

We proceed by calculating the distances between the points along each axis, and center the picking point on the axis where the shortest distance was found.



Finding the point on the original image

We found the picking point on the cropped image, but of course the output should be the point on the original image. To do this we revert all the changes explained in the 3rd slide, and save the point coordinates.



Visualizing all the picking points on the original image

Since we calculate the picking point for all the objects in an image, we can use that to visualize all of them on the original RGB image.



Choosing the object to pick up first

Along with the picking point position, we can also return the average depth of the object, so we can know which object to pick up first.



Performance

Here we can see the performance for every object's class.

```
Class Tubo
Average time: 34.691737 ms
Min time: 33.280790 ms
Max time: 37.612764 ms
Class Ugello
Average time: 34.268280 ms
Min time: 32.689157 ms
Max time: 35.894614 ms
Class Dado
Average time: 33.408813 ms
Min time: 31.990654 ms
Max time: 36.638463 ms
Class Vite
Average time: 34.749525 ms
Min time: 32.465858 ms
Max time: 42.629756 ms
Class Chiave
Average time: 38.890260 ms
Min time: 32.983561 ms
Max time: 45.617835 ms
```

Implementation with ROS

After implementing the picking point algorithm, we proceeded to integrate it into ROS nodes to enable its use in real-world scenarios.

We defined three nodes: applicant, picking point handler and the receiver



Applicant

The applicant loads the mask, the depth map and the original rgb image from the disk and publishes them. In our case it iterates on all the masks and sends them.

Picking Point Handler

The picking point handler receives all the images published by the applicant node, and gives them to the PickingPoint object, which will find the picking point and return all the relative data.

Receiver

The receiver receives all the data calculated and published by the picking point handler node and prints them.

```
[fabio@arch ROS]$ ros2 run ROS_PickingPoint applicant
Path: assets/masks/black_0/black_0_mask_6_3.png
[INFO] [1720021054.084694462] [applicant]: Image size: 640x480
[INFO] [1720021054.084770565] [applicant]: Image channels: 3
[INFO] [1720021054.084782848] [applicant]: Depth image size: 640x480
[INFO] [1720021054.084791665] [applicant]: Depth image channels: 3
Press any key to continue (q to quit) ...

[fabio@arch ROS]$ ros2 run ROS_PickingPoint pickingPointHandler
[INFO] [1720021053.757528573] [picking_point_handler]: PickingPointHandler node has been started
[INFO] [1720021054.076326250] [picking_point_handler]: Image size: 640x480
[INFO] [1720021054.076393166] [picking_point_handler]: Image channels: 3
[INFO] [1720021054.099120023] [picking_point_handler]: Image size: 640x480
[INFO] [1720021054.099217727] [picking_point_handler]: Image channels: 3
[INFO] [1720021054.106711165] [picking_point_handler]: Image size: 640x480
[INFO] [1720021054.106774875] [picking_point_handler]: Image channels: 3

[fabio@arch ROS]$ ros2 run ROS_PickingPoint receiver
[INFO] [1720021048.994100803] [receiver]: Receiver node has been started.
[INFO] [1720021054.124927793] [receiver]: Position: (199.00, 298.00)
[INFO] [1720021054.124980943] [receiver]: 1) Opening 10.00, Angle 14.04
[INFO] [1720021054.124995811] [receiver]: 2) Opening 14.00, Angle 104.04
[INFO] [1720021054.125018063] [receiver]: Object AVG Depth: 359.33
```

Outputs

Here we can see the two openings drawn on the mask.



Links

Picking Point Repo:

<https://github.com/Fxby16/PickingPoint>

ROS Repo:

https://github.com/Fxby16/ROS_PickingPoint